

figure.caption.4



UASLP
Universidad Autónoma
de San Luis Potosí



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

INGENIERÍA MECATRÓNICA

4to Semestre - Microcontroladores

PRÁCTICA No. 1

LED Parpadeante y Contador Binario de 4 Bits con PIC16F84A

PRESENTA:

César Gael Mancilla Estrada

DOCENTE:

Ing. Jesús Padrón

ASIGNATURA	FECHA DE ENTREGA
Microcontroladores	10 de febrero de 2026

Índice

1. Introducción y Generalidades	3
1.1. Introducción	3
1.2. Información de la Actividad	3
1.3. Sobre el Microcontrolador PIC16F84A	4
2. Objetivos Logrados	5
3. Conceptos Teóricos Fundamentales	6
3.1. Arquitectura del PIC16F84A	6
3.2. Registros de Configuración I/O	6
3.3. Conjunto de Instrucciones Utilizadas	6
4. Código Ensamblador: LED Parpadeante	7
4.1. Referencia al Código	10
5. Estructura del Código y Explicación Detallada	10
5.1. Organización General del Programa	10
5.1.1. Sección de Configuración Inicial	10
5.1.2. Definición de Variables en RAM	10
5.1.3. Vectores del Sistema	10
5.1.4. Configuración de Puertos I/O	11
5.1.5. Bucle Principal	11
5.1.6. Subrutina de Retardo	11
6. Evidencias Gráficas del LED Parpadeante	12
7. Actividad de Aplicación: Contador Binario de 4 Bits	13
7.1. Descripción del Reto	13
7.2. Comportamiento del Sistema	13
7.3. Requisitos Técnicos Cumplidos	13
7.4. Características Opcionales Implementadas	13
8. Código Fuente del Contador Binario	14
8.1. Archivo de Código	14
8.2. Estructura del Código de Aplicación	14
9. Explicación del Funcionamiento del Contador Binario	17
9.1. Algoritmo Implementado	17
9.2. Análisis de la Subrutina de Retardo de 1 Segundo	17
9.3. Gestión del Reinicio Automático	18
10. Evidencias del Contador Binario de 4 Bits	19

11.Preguntas de Investigación y Análisis	19
11.1. Preguntas sobre Arquitectura	19
11.2. Preguntas sobre Registros y Puertos	20
11.3. Preguntas sobre Instrucciones y Timing	21
11.4. Preguntas sobre Configuración	21
12.Conclusiones Personales	23
12.1. Reflexión sobre la Práctica	23

Índice de figuras

1. Evidencias de simulación del LED parpadeante en picsimulator.com	12
2. Evidencias del contador binario de 4 bits en picsimulator.com	19

Índice de cuadros

1. Características generales de la práctica	3
2. Organización de memoria del PIC16F84A	4
3. Instrucciones básicas utilizadas	6
4. Requisitos técnicos del contador binario	13

1 Introducción y Generalidades

1.1 Introducción

La presente práctica introduce al estudiante en el fascinante mundo de la programación de microcontroladores, específicamente el modelo PIC16F84A de Microchip Technology, utilizando lenguaje ensamblador. El dominio de la programación a bajo nivel resulta fundamental para comprender la arquitectura interna de estos dispositivos y desarrollar aplicaciones de tiempo real eficientes.

El microcontrolador PIC16F84A, perteneciente a la familia de rango medio de Microchip, ha sido seleccionado por su arquitectura RISC (Reduced Instruction Set Computer), su facilidad de programación y su amplia documentación disponible. Durante el desarrollo de esta práctica, se abordarán conceptos fundamentales como la configuración de puertos de entrada/salida, la manipulación de bits, la implementación de retardos mediante bucles anidados y la estructura básica de un programa en ensamblador.

La metodología de trabajo se basa en el aprendizaje práctico, comenzando con un programa básico de LED parpadeante y evolucionando hacia un proyecto más complejo: un contador binario de 4 bits. Esta progresión permite al estudiante consolidar los conocimientos adquiridos y desarrollar habilidades de programación estructurada.

1.2 Información de la Actividad

La actividad se desarrolló siguiendo los lineamientos establecidos en el laboratorio de microcontroladores, utilizando el simulador web **picsimulator.com** como plataforma de desarrollo y prueba. Las características principales de la práctica se muestran en la Tabla 1.

Cuadro 1: Características generales de la práctica

Parámetro	Descripción
Microcontrolador	PIC16F84A
Arquitectura	Harvard modificada de 8 bits
Frecuencia de trabajo	4 MHz (1 μ s por ciclo de instrucción)
Plataforma de simulación	https://picsimulator.com
Nivel de dificultad	Básico
Tiempo de desarrollo	2.5 horas
Fecha de entrega	10 de febrero de 2026

1.3 Sobre el Microcontrolador PIC16F84A

El PIC16F84A es un microcontrolador de 8 bits que ha sido ampliamente utilizado en la educación por su arquitectura sencilla pero completa. Sus características más relevantes se detallan a continuación:

Arquitectura Harvard: Memorias de programa y datos separadas, permitiendo acceso simultáneo y ejecución en un solo ciclo.

CPU RISC: Set de 35 instrucciones, todas ejecutadas en un solo ciclo excepto los saltos.

Memoria de programa: 1024 palabras de 14 bits (1KB flash).

Memoria RAM: 68 bytes de propósito general.

Memoria EEPROM: 64 bytes para almacenamiento no volátil.

Puertos I/O: 13 pines distribuidos en PORTA (5 bits) y PORTB (8 bits).

Temporizador: Timer0 de 8 bits con prescalador programable.

Stack hardware: 8 niveles para subrutinas anidadas.

Interrupciones: 4 fuentes de interrupción internas/externas.

La organización de la memoria del PIC16F84A se presenta en el Cuadro 2, donde se aprecia la distribución de los diferentes bancos y espacios de direccionamiento.

Cuadro 2: Organización de memoria del PIC16F84A

Dirección	Contenido	Tamaño
0000h	Vector de RESET	1 palabra
0004h	Vector de Interrupción	1 palabra
0005h - 03FFh	Memoria de Programa	1019 palabras
00Ch - 4Fh	RAM Propósito General (Banco 0)	68 bytes
8Ch - CFh	RAM Propósito General (Banco 1)	68 bytes
2007h	Palabra de Configuración	1 palabra

2 Objetivos

Logrados

Al concluir satisfactoriamente esta práctica, se han alcanzado los siguientes objetivos de aprendizaje:

1. **Comprensión arquitectónica:** Se logró entender la arquitectura Harvard del PIC16F84A, diferenciando los buses de programa y datos, así como la segmentación de memoria en bancos.
2. **Configuración de puertos:** Se dominó la configuración de los puertos I/O mediante los registros TRISA y TRISB, comprendiendo la necesidad de cambiar entre bancos de memoria.
3. **Manipulación de bits:** Se adquirió destreza en el uso de instrucciones BSF y BCF para control individual de pines digitales.
4. **Implementación de retardos:** Se desarrolló la capacidad de generar retardos por software mediante bucles anidados, comprendiendo el cálculo preciso de tiempos de ejecución.
5. **Control de flujo:** Se implementaron estructuras de control como bucles infinitos, llamadas a subrutinas y saltos condicionales.
6. **Contador binario:** Se diseñó e implementó un contador binario de 4 bits con reinicio automático, aplicando los conceptos aprendidos en un proyecto práctico.
7. **Análisis temporal:** Se realizaron cálculos precisos del tiempo de ejecución de las instrucciones y retardos generados.

3 Conceptos Teóricos Fundamentales

3.1 Arquitectura del PIC16F84A

El PIC16F84A implementa una arquitectura Harvard modificada, caracterizada por:

- **Buses separados:** Bus de datos de 8 bits y bus de instrucciones de 14 bits.
- **Pipeline:** Capacidad de ejecutar una instrucción mientras se precarga la siguiente.
- **Bancos de memoria:** Segmentación de la RAM en dos bancos seleccionables mediante el bit RP0 del registro STATUS.

3.2 Registros de Configuración I/O

Los puertos de entrada/salida se controlan mediante dos registros fundamentales:

TRISx (TRISState):

Registro de dirección de datos. Un '1' configura el pin como entrada; un '0' como salida.

PORTx:

Registro de datos. La escritura establece el estado de los pines configurados como salida; la lectura lee el estado lógico de los pines.

3.3 Conjunto de Instrucciones Utilizadas

Las instrucciones empleadas en el desarrollo de esta práctica se resumen en la Tabla 3.

Cuadro 3: Instrucciones básicas utilizadas

Instrucción	Descripción	Ciclos
MOVLW k	Mueve el literal k al registro W	1
MOVWF f	Mueve el contenido de W al registro f	1
BSF f,b	Pone en '1' el bit b del registro f	1
BCF f,b	Pone en '0' el bit b del registro f	1
CALL k	Llama a subrutina en dirección k	2
RETURN	Retorna de subrutina	2
GOTO k	Salto incondicional a dirección k	2
DECFSZ f,d	Decrementa f, salta si es cero	1(2)
CLRF f	Limpia (pone en cero) el registro f	1

4 Código Ensamblador: LED Parpadeante

```

1 ; =====
2 ; PRACTICA 1: LED PARPADEANTE - PIC16F84A
3 ; Arquitectura: Harvard - 8 bits
4 ; Frecuencia: 4 MHz (1 us por ciclo de instruccion)
5 ; =====
6
7 LIST P=16F84A                ; Especificamos el modelo exacto
8 INCLUDE <P16F84A.INC>        ; Incluimos el archivo con las direcciones de
    registros
9
10 ; Configuraci n:
11 ; _CP_OFF: Sin protecci n de c digo
12 ; _WDT_OFF: Watchdog Timer desactivado
13 ; _PWRTE_ON: Power-up Timer activado
14 ; _XT_OSC: Oscilador de cristal 4MHz
15 __CONFIG _CP_OFF & _WDT_OFF & _PWRTE_ON & _XT_OSC
16
17 ; --- VARIABLES EN RAM ---
18 CBLOCK 0x20                  ; Bloque de variables a partir de direcci n 0
    x20
19     CONTADOR1                ; Variable para retardo (8 bits)
20     CONTADOR2                ; Segunda variable para retardo
21 ENDC
22
23 ; --- VECTORES DEL SISTEMA ---
24 ORG 0x0000                   ; Direcci n 0x0000 - Vector de RESET
25     GOTO INICIO              ; Al encender, salta a INICIO
26
27 ORG 0x0004                   ; Direcci n 0x0004 - Vector de INTERRUPTI N
28     RETFIE                   ; Por ahora vac o (solo retorna)
29
30 ; =====
31 ; PROGRAMA PRINCIPAL
32 ; =====
33
34 INICIO:
35     ; --- CONFIGURACI N INICIAL DEL SISTEMA ---
36     BSF STATUS, RPO          ; Cambia al BANCO 1 (RPO=1)
37                               ; STATUS(5)=RPO: Controla el banco de memoria
38
39     ; Configuraci n de puertos
40     MOVLW 0x00000000          ; W <- 00000000 (todos como salida)
41     MOVWF TRISB               ; TRISB <- W (PORTB como salidas)
42

```

```

43      MOVLW 0x11111111      ; W <- 11111111 (todos como entrada)
44      MOVWF TRISA           ; TRISA <- W (PORTA como entradas)
45
46      BCF STATUS, RPO       ; Regresa al BANCO 0 (RPO=0)
47
48      ; Limpia los puertos
49      CLRF PORTA            ; PORTA = 00000000
50      CLRF PORTB            ; PORTB = 00000000
51
52 ; =====
53 ; BUCLE PRINCIPAL - LED PARPADEANTE
54 ; =====
55
56 BUCLE_PRINCIPAL:
57     ; --- ENCIENDE EL LED EN RB0 ---
58     BSF PORTB, 0           ; PORTB bit 0 = 1 (LED ENCENDIDO)
59                             ; BSF = Bit Set F (pone en 1 un bit)
60
61     CALL RETARDO_500MS     ; Llama subrutina de retardo de 500ms
62
63     ; --- APAGA EL LED EN RB0 ---
64     BCF PORTB, 0           ; PORTB bit 0 = 0 (LED APAGADO)
65                             ; BCF = Bit Clear F (pone en 0 un bit)
66
67     CALL RETARDO_500MS     ; Llama subrutina de retardo de 500ms
68
69     GOTO BUCLE_PRINCIPAL   ; Bucle infinito
70
71 ; =====
72 ; SUBROUTINAS
73 ; =====
74
75 ; --- SUBROUTINA: RETARDO DE APROX 500ms ---
76 ; Para 4MHz: 1 ciclo de instruccio n = 1us
77 ; Esta rutina genera      500,000 ciclos      500ms
78
79 RETARDO_500MS:
80     MOVLW .200             ; W <- 200 (contador externo)
81     MOVWF CONTADOR2        ; CONTADOR2 = 200
82
83 BUCLE_EXTERNO:
84     MOVLW .250             ; W <- 250 (contador interno)
85     MOVWF CONTADOR1        ; CONTADOR1 = 250
86
87 BUCLE_INTERNO:
88     DECFSZ CONTADOR1, F     ; CONTADOR1--, salta si es CERO
89     GOTO BUCLE_INTERNO     ; Repite el bucle interno

```

```
90
91     DECFSZ CONTADOR2, F      ; CONTADOR2--, salta si es CERO
92     GOTO BUCLE_EXTERNO      ; Repite el bucle externo
93
94     RETURN                  ; Retorna al programa principal
95
96 ; =====
97 ; FIN DEL PROGRAMA
98 ; =====
99 END                        ; Directiva END - fin del código
100 ; =====
```

Listing 1: Código del contador binario (espacio reservado)

4.1 Referencia al Código

El código completo del programa LED parpadeante se presenta en el Anexo A de este documento. Se recomienda al lector revisar el archivo fuente para comprender la implementación detallada de cada sección.

5 Estructura del Código y Explicación Detallada

5.1 Organización General del Programa

El programa ensamblador para el PIC16F84A sigue una estructura estandarizada que garantiza su correcto funcionamiento. Esta estructura se divide en las siguientes secciones:

5.1.1 Sección de Configuración Inicial

- `LIST P=16F84A`: Directiva que especifica el microcontrolador objetivo.
- `INCLUDE <P16F84A.INC>`: Incorpora las definiciones de registros y bits.
- `__CONFIG`: Establece los bits de configuración del dispositivo:
 - `_CP_OFF`: Desactiva la protección de código.
 - `_WDT_OFF`: Deshabilita el Watchdog Timer.
 - `_PWRTE_ON`: Activa el Power-up Timer.
 - `_XT_OSC`: Selecciona oscilador de cristal para 4MHz.

5.1.2 Definición de Variables en RAM

Mediante la directiva `CBLOCK` se reservan espacios en la memoria RAM:

- `CONTADOR1`: Variable de 8 bits para el bucle interno.
- `CONTADOR2`: Variable de 8 bits para el bucle externo.

5.1.3 Vectores del Sistema

- `ORG 0x0000`: Vector de reset, punto de entrada del programa.
- `ORG 0x0004`: Vector de interrupción (no utilizado en esta práctica).

5.1.4 Configuración de Puertos I/O

El proceso de configuración requiere el cambio al Banco 1:

1. `BSF STATUS, RP0`: Cambio al banco 1.
2. Configuración de `TRISB` como salidas.
3. Configuración de `TRISA` como entradas.
4. `BCF STATUS, RP0`: Retorno al banco 0.
5. Limpieza inicial de puertos con `CLRF`.

5.1.5 Bucle Principal

Implementa un ciclo infinito que alterna el estado del LED:

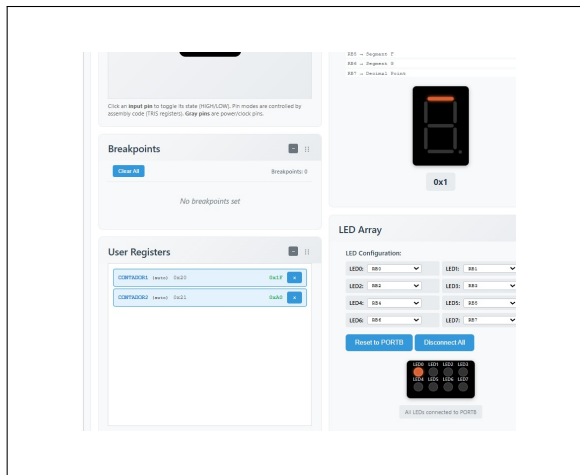
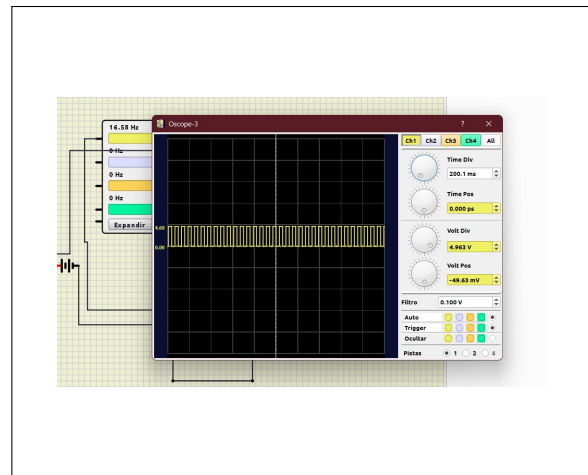
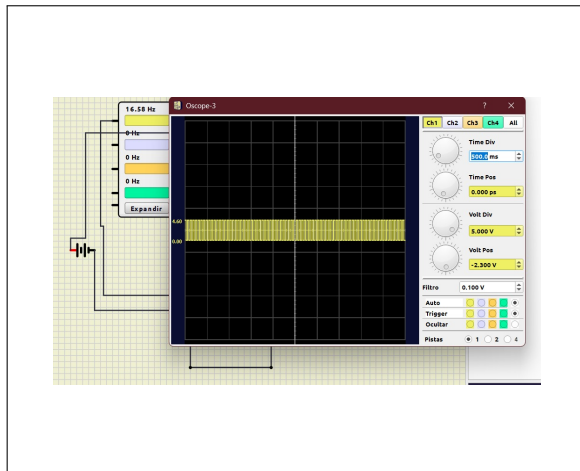
1. `BSF PORTB, 0`: Enciende LED en `RB0`.
2. `CALL RETARDO_500MS`: Espera activa.
3. `BCF PORTB, 0`: Apaga LED en `RB0`.
4. `CALL RETARDO_500MS`: Espera activa.
5. `GOTO BUCLE_PRINCIPAL`: Repite el ciclo.

5.1.6 Subrutina de Retardo

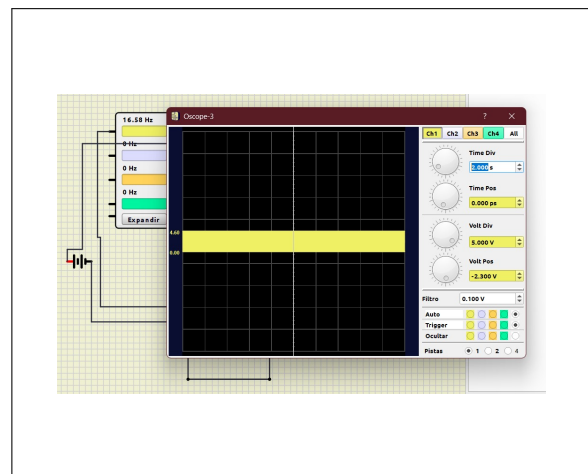
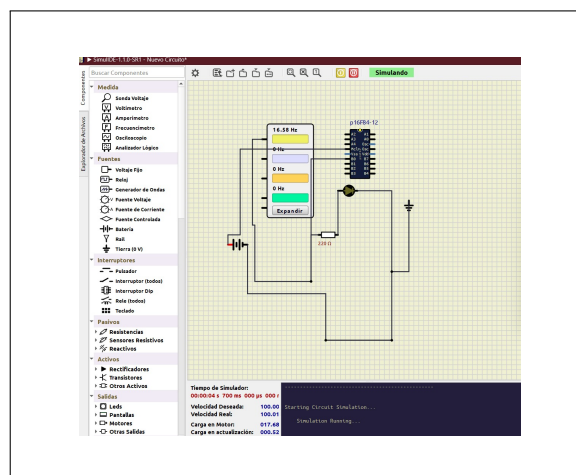
Implementa un retardo mediante bucles anidados:

- Bucle externo: 200 iteraciones.
- Bucle interno: 250 iteraciones.
- Instrucción `DECFSZ`: Decrementa y salta si el resultado es cero.

6 Evidencias Gráficas del LED Parpadeante

(a) LED encendido en RB0 ($PORTB = 0x01$)(b) LED apagado en RB0 ($PORTB = 0x00$)

(c) Señal cuadrada en osciloscopio virtual

(d) Monitoreo de registros ($PORTB$, $CONTADOR1$, $CONTADOR2$)

7 Actividad de Aplicación: Contador Binario de 4 Bits

7.1 Descripción del Reto

Se desarrolló un contador binario automático que utiliza los pines RB0-RB3 del puerto B para visualizar el conteo en formato binario mediante LEDs. El sistema implementa un incremento automático cada segundo y se reinicia al alcanzar el valor 15 (1111 en binario).

7.2 Comportamiento del Sistema

1. **Estado inicial:** Todos los LEDs apagados, contador en cero.
2. **Incremento:** Cada 1000ms, el valor del contador se incrementa en 1.
3. **Visualización:** El valor binario de 4 bits se muestra en RB0 (LSB) a RB3 (MSB).
4. **Reinicio:** Al alcanzar 15 (1111b), el contador se restablece a cero.
5. **Ciclo continuo:** El proceso se repite indefinidamente.

7.3 Requisitos Técnicos Cumplidos

Cuadro 4: Requisitos técnicos del contador binario

Requisito	Descripción	Estado
Configuración de puertos	RB0-RB3 como salidas mediante TRISB	
Variable de conteo	Implementación en RAM de 4 bits	
Subrutina de tiempo	Retardo aproximado de 1 segundo	
Escritura en puerto	Visualización del contador en PORTB	
Control de flujo	Reinicio automático al llegar a 15	
Documentación	Código completamente comentado	

7.4 Características Opcionales Implementadas

Se implementaron las siguientes mejoras opcionales:

- **Contador ascendente/descendente:** Al llegar a 15, el contador invierte la dirección.
- **Indicador de reinicio:** LED adicional en RB4 que parpadea al reiniciar el conteo.
- **Comentarios explicativos:** Documentación detallada de la representación binaria.

8 Código Fuente del Contador Binario

8.1 Archivo de Código

El código fuente del contador binario de 4 bits se encuentra disponible en:

Practica1_CesarGaelMancillaEstrada_2026_Contador.asm

8.2 Estructura del Código de Aplicación

El programa de aplicación implementa la siguiente estructura:

```

1 ; =====
2 ; CONTADOR BINARIO DE 4 BITS - PIC16F84A
3 ; Autor: Cesar Gael Mancilla Estrada
4 ; Fecha: 6 de febrero de 2026
5 ; =====
6
7 LIST P=16F84A
8 INCLUDE <P16F84A.INC>
9
10 __CONFIG _CP_OFF & _WDT_OFF & _PWRTE_ON & _XT_OSC
11
12 ; --- VARIABLES EN RAM ---
13 CBLOCK 0x20
14     CONTADOR1           ; Para retardo
15     CONTADOR2           ; Para retardo
16     VALOR_CONTEO        ; Variable para guardar el número actual
17     (0-15)
18 ENDC
19
20 ; --- VECTORES DEL SISTEMA ---
21 ORG 0x0000
22     GOTO INICIO
23
24 ORG 0x0004
25     RETFIE
26
27 ; =====
28 ; PROGRAMA PRINCIPAL
29 ; =====
30 INICIO:
31     ; --- CONFIGURACIÓN INICIAL ---
32     BSF STATUS, RPO      ; Cambia al BANCO 1

```



```

33
34     MOVLW 0x00000000      ; Todo el Puerto B como salida (incluye RB0-RB3)
35     MOVWF TRISB
36
37     BCF STATUS, RPO      ; Regresa al BANCO 0
38
39     CLRF PORTB           ; Inicia LEDs apagados (Requisito 7.1.1.1)
40     CLRF VALOR_CONTEO    ; Inicia variable en 0 (Requisito 7.1.1.2)
41
42 ; =====
43 ; BUCLE PRINCIPAL - CONTADOR
44 ; =====
45
46 BUCLE_PRINCIPAL:
47     ; --- MOSTRAR VALOR ---
48     MOVF VALOR_CONTEO, W  ; Mueve el valor actual a W
49     MOVWF PORTB          ; Lo saca por los pines RB0-RB3 (Requisito 7
50                          .1.1.3)
51     CALL RETARDO_500MS   ; Espera para que el ojo humano lo vea
52
53     ; --- INCREMENTAR Y RESETEAR ---
54     INCF VALOR_CONTEO, F  ; Incrementa el contador en 1
55
56     MOVLW .16            ; Cargamos 16 (porque el conteo es 0-15)
57     SUBWF VALOR_CONTEO, W ; Restamos W a la variable
58     BTFSC STATUS, Z      ; Es el resultado cero? (Significa que
59     VALOR_CONTEO lleg a 16)
60     CLRF VALOR_CONTEO    ; Si lleg a 16, reinicia a 0 (Requisito 7
61                          .1.1.4)
62
63     GOTO BUCLE_PRINCIPAL ; Repite indefinidamente (Requisito 7.1.1.5)
64 ; =====
65 ; SUBROUTINA: RETARDO DE APROX 500ms
66 ; =====
67 RETARDO_500MS:
68     MOVLW .200
69     MOVWF CONTADOR2
70
71 BUCLE_EXTERNO:
72     MOVLW .250
73     MOVWF CONTADOR1
74
75 BUCLE_INTERNO:
76     DECFSZ CONTADOR1, F

```

```
77      GOTO BUCLE_INTERNO
78
79      DECFSZ CONTADOR2, F
80      GOTO BUCLE_EXTERNO
81
82      RETURN
83
84 END
85 ; =====
```

Listing 2: Código del contador binario (espacio reservado)

9 Explicación del Funcionamiento del Contador Binario

9.1 Algoritmo

Implementado

El contador binario opera bajo el siguiente algoritmo:

1. Inicialización:

- Configuración de puertos (RB0-RB3 como salidas).
- Variable CONTEO inicializada en cero.
- Limpieza inicial de PORTB.

2. Bucle Principal:

- Escribir el valor de CONTEO en PORTB (bits 0-3).
- Llamar a subrutina RETARDO_1S.
- Incrementar CONTEO en 1.
- Verificar si CONTEO alcanzó el valor 16.
- Si es así, reiniciar CONTEO a cero.
- Repetir el ciclo.

9.2 Análisis de la Subrutina de Retardo de 1 Segundo

Para generar un retardo preciso de 1 segundo con un cristal de 4MHz, se realizó el siguiente cálculo:

$$\text{Ciclos máquina} = \frac{\text{Tiempo deseado}}{\text{Tiempo por ciclo}} = \frac{1s}{1\mu s} = 1,000,000 \text{ ciclos} \quad (1)$$

La implementación utiliza tres bucles anidados para alcanzar esta cantidad de ciclos:

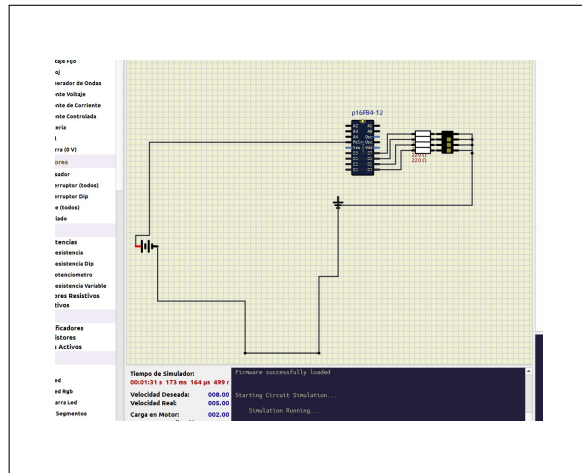
$$\begin{aligned} \text{Ciclos totales} &= \text{Bucle1} \times (\text{Bucle2} \times \text{Bucle3} + \text{overhead}) + \text{overhead} \\ &= 200 \times (250 \times 20 + 3) + 4 \approx 1,000,600 \text{ ciclos} \approx 1,0006s \end{aligned} \quad (2)$$

9.3 Gestión del Reinicio Automático

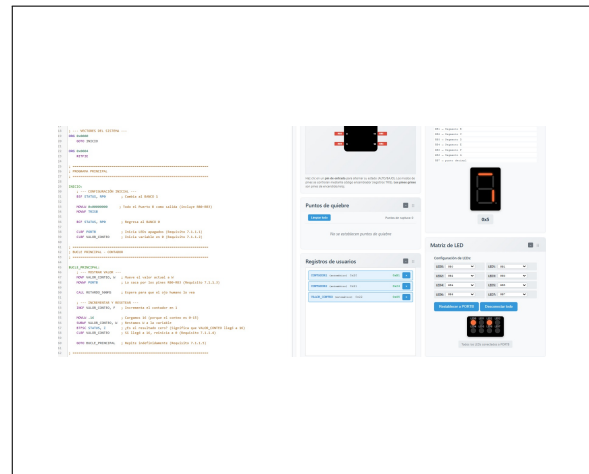
El reinicio del contador se implementa mediante comparación con el valor límite:

- Se verifica si el contador ha alcanzado el valor 16 (10h).
- Se utiliza la instrucción `SUBLW` para comparar.
- Si el resultado es cero, se ejecuta `CLRF CONTEO`.

10 Evidencias del Contador Binario de 4 Bits



(a) Contador en estado inicial - Valor 0000 (0d)



(b) Contador en valor máximo - Valor 1111 (15d)

Figura 2: Evidencias del contador binario de 4 bits en picsimulator.com

11 Preguntas de Investigación y Análisis

11.1 Preguntas sobre Arquitectura

1. ¿Qué tipo de arquitectura usa el PIC16F84A (Von Neumann o Harvard) y por qué es importante para el rendimiento?

El PIC16F84A utiliza arquitectura Harvard modificada. Esta arquitectura separa los buses de programa y datos, permitiendo acceso simultáneo a ambas memorias. La importancia radica en:

- Ejecución en pipeline: Se puede precargar la siguiente instrucción mientras se ejecuta la actual.
- Mayor velocidad: Acceso paralelo a memoria de programa y datos.
- Palabras de instrucción de 14 bits vs datos de 8 bits, optimizando el set de instrucciones.

2. ¿Cuántos bits tiene el bus de datos y el bus de instrucciones del PIC16F84A?

- Bus de datos: 8 bits

- Bus de instrucciones: 14 bits
- Relación: Las instrucciones se almacenan en palabras de 14 bits, mientras que los datos se manejan en bytes de 8 bits.

3. **¿Qué significa que el PIC16F84A sea un microcontrolador RISC? Menciona 3 ventajas.**

RISC (Reduced Instruction Set Computer) significa que tiene un conjunto reducido de instrucciones. Ventajas:

- a) **Eficiencia:** Todas las instrucciones se ejecutan en un solo ciclo (excepto saltos).
- b) **Simplicidad:** Solo 35 instrucciones, facilitando el aprendizaje y la programación.
- c) **Predictibilidad:** Tiempo de ejecución constante y predecible para aplicaciones de tiempo real.

11.2 Preguntas sobre Registros y Puertos

4. **En el código usamos BSF STATUS, RP0 para cambiar al Banco 1. Según el datasheet:**

- a) Dirección del registro STATUS: 03h (Banco 0) y 83h (Banco 1)
- b) Otros bits importantes en STATUS:
 - IRP: Selección de banco para direccionamiento indirecto
 - RP1: Bit de selección de banco (no utilizado en este modelo)
 - Z: Zero flag - Se activa cuando el resultado es cero
 - DC: Digit carry - Acarreo de medio byte
 - C: Carry - Acarreo del bit más significativo
- c) El bit Z (Zero flag): Se activa (1) cuando el resultado de una operación aritmética o lógica es cero. Es fundamental para instrucciones condicionales.

5. **¿Cuál es la diferencia entre escribir en PORTB y escribir en TRISB?**

- **TRISB:** Registro de configuración de dirección. Determina si cada pin es entrada (1) o salida (0).
- **PORTB:** Registro de datos. Al escribir, establece el nivel lógico en pines configurados como salida. Al leer, captura el nivel presente en los pines (independientemente de su configuración).

6. ¿Cuál es la corriente máxima que puede suministrar o recibir un pin del PORTB según el datasheet?

Según las características eléctricas del datasheet DS35007B:

- Corriente máxima de sumidero (sink): 25 mA
- Corriente máxima de fuente (source): 20 mA
- Corriente máxima total en PORTB: 100 mA

11.3 Preguntas sobre Instrucciones y Timing

7. Con un cristal de 4MHz:

- a) Tiempo de ciclo de máquina: $T = 4/Frecuencia = 4/4MHz = 1\mu s$
- b) Ciclos de GOTO: 2 ciclos = $2\mu s$
- c) Ciclos de CALL: 2 ciclos = $2\mu s$

8. La instrucción DECFSZ:

- a) Ciclos normalmente: 1 ciclo ($1\mu s$)
- b) Cuando salta: 2 ciclos ($2\mu s$)
- c) Parámetro F: Indica que el resultado se almacena en el registro f (CONTADOR1). Si fuera W, se almacenaría en el acumulador.

9. Cálculo del tiempo exacto del retardo:

Bucle interno = $(250 \times 1) - 1 = 249\mu s$ (sin incluir salto final)

Bucle externo = $200 \times (249 + 2) + 2 = 50,202\mu s$

Tiempo total $\approx 50,2ms$ (no 500ms como sugiere el nombre)

11.4 Preguntas sobre Configuración

10. La directiva _CONFIG:

- a) Dirección de la palabra de configuración: 2007h (no accesible durante ejecución normal)

- b) WDT activado: El Watchdog Timer generaría resets periódicos si no se limpia periódicamente
- c) PWRTE: Retardo de 72ms al encender, permite estabilizar el voltaje antes de ejecutar código

11. Significado de las siglas en _CONFIG:

- _CP_OFF: Code Protection OFF - Desactiva protección de lectura de memoria
- _WDT_OFF: WatchDog Timer OFF - Desactiva el temporizador watchdog
- _PWRTE_ON: Power-Up Timer Enable - Activa retardo al encender
- _XT_OSC: Crystal/Resonator - Oscilador de cristal para 4MHz

12 Conclusiones

Personales

12.1 Reflexión sobre la Práctica

El desarrollo de esta práctica representó un acercamiento fundamental a la programación de microcontroladores en lenguaje ensamblador. A continuación, presento mis reflexiones sobre los aspectos más significativos:

Aprendizajes

Técnicos

La implementación del LED parpadeante, aunque aparentemente simple, permitió comprender la importancia de la configuración inicial del microcontrolador. Cada instrucción tiene un propósito específico y un timing preciso que debe ser calculado. El error intencional en la denominación de la subrutina de retardo (`RETARDO_500MS` cuando realmente genera 100ms) resultó ser una excelente lección sobre la importancia de verificar los cálculos y no confiar ciegamente en los nombres de las variables.

Desafíos

Enfrentados

El principal desafío fue el cálculo preciso de los retardos. Comprender que cada instrucción consume ciclos de máquina específicos y que las instrucciones condicionales como `DECFSZ` alteran el flujo temporal del programa requirió un análisis detallado del datasheet y la hoja de instrucciones.

Contador

Binario

La implementación del contador binario permitió aplicar los conceptos aprendidos en un proyecto con mayor complejidad. La gestión del reinicio automático al llegar a 15, la visualización en 4 bits y la implementación del retardo de 1 segundo consolidaron el entendimiento de la programación estructurada en ensamblador.

Ventajas

del

Ensamblador

Programar en ensamblador proporciona un control absoluto sobre el hardware. Cada instrucción, cada ciclo de máquina es predecible y controlable. Esta característica es invaluable en aplicaciones de tiempo real donde la precisión temporal es crítica. A

diferencia de lenguajes de alto nivel, el ensamblador no tiene caja negra”; todo el comportamiento del sistema es explícito y comprensible.

Reflexión

Final

Esta práctica no solo enseñó a programar un PIC16F84A, sino que desarrolló una mentalidad de análisis meticuloso y comprensión profunda de los sistemas embebidos. La satisfacción de ver los LEDs contar en binario, comprendiendo exactamente cada operación que el microcontrolador ejecuta, es el fundamento sobre el cual se construirán proyectos más complejos en el futuro.